

Problem Scuderia

Autor problemă:

Scurtă descriere a problemei

Se dau n valori care trebuie aranjate într-o matrice cu k coloane. Trebuie să determinăm valoarea maximă a unei sume calculate printr-o anumită formulă care combină sume de linii, coloane și regiuni dreptunghiulare, folosind ferestre glisante de dimensiuni p și q .

Idei principale

Aranjarea datelor într-o matrice 2D

Primul pas este transformarea celor n valori într-o matrice bidimensională cu k coloane. Folosind formula de indexare $i \cdot k + j$, unde i este indicele liniei și j este indicele coloanei, putem accesa elementele ca și cum ar fi într-o matrice 2D.

Offset-ul calculat ca $k - r$ permite poziționarea corectă a datelor în matrice, astfel încât calculele ulterioare să funcționeze corect până la marginile definite de p și r .

Sume prefix 2D

Cheia soluției este construirea unei table de sume prefix 2D, unde fiecare element conține suma tuturor elementelor din dreapta-sus față de el:

$$\text{prefix}(i, j) = a_{i,j} + \text{prefix}(i - 1, j) + \text{prefix}(i, j - 1) - \text{prefix}(i - 1, j - 1)$$

Această formulă permite calcularea sumei oricărui sub-dreptunghi (x_1, y_1) la (x_2, y_2) în timp $O(1)$:

$$\text{sum}(x_1, y_1, x_2, y_2) = \text{prefix}(x_2, y_2) - \text{prefix}(x_1 - 1, y_2) - \text{prefix}(x_2, y_1 - 1) + \text{prefix}(x_1 - 1, y_1 - 1)$$

Funcția de conveniență $s(i, j)$ gestionează automat cazurile de graniță, returnând 0 pentru indici negativi.

Formulele de calcul

Soluția explorează diferite configurații geometrice pentru a găsi suma maximă:

Suma de linie: Pentru un interval vertical de q elemente până la linia i :

$$\text{lineSum}(i) = s(i, k - 1) - s(i - q, k - 1)$$

Suma de coloană: Pentru un interval orizontal de p elemente până la coloana j :

$$\text{colSum}(j) = s(\text{lastLine}, j) - s(\text{lastLine}, j - p)$$

Suma intersecției: Folosind principiul incluziun-excluziunii pentru regiunea comună:

$$\text{intersectSum} = s(i, j) - s(i - q, j) + s(i - q, j - p) - s(i, j - p)$$

Suma finală pentru această configurație este:

$$\text{lineSum} + \text{colSum} - \text{intersectSum}$$

Acest lucru reprezintă suma tuturor elementelor din cele trei regiuni distincte (linia albastră, coloana roșie, minus suprapunerea).

Fereastra glisantă

Algoritmul iterează prin toate configurațiile posibile folosind două bucle imbricate:

1. Prima buclă parcurge liniile de la q la lastLine
2. A doua buclă parcurge coloanele relevante

Pentru fiecare poziție, se calculează suma combinată și se actualizează maximul global. Complexitatea finală este $O(n)$, unde n este numărul de elemente, datorită calculului $O(1)$ al fiecărei sume prin tabla prefix.

Implementare

```
#include <fstream>
#include <iostream>
#include <math.h>
#include <limits.h>

using namespace std;

const int NMAX = 1e6 + 1;
const int OFFSET_MAX = 1e5 + 1;
int a[NMAX + 2 * OFFSET_MAX];
int n, k, p, q, r;

int s(int i, int j)
{
    if (i < 0 || j < 0)
        return 0;
    return a[i * k + j];
}

int getCoords(int x, int y)
{
    return x * k + y;
}

int main()
{
```

```

ifstream fin("scuderia.in");
ofstream fout("scuderia.out");

fin >> n >> k >> p >> q >> r;

if (p > k)
    p = k;

int offset = k - r, totalSum = 0;

for (int i = 0; i < n; ++i)
{
    fin >> a[i + offset];
    totalSum += a[i + offset];
}

n += offset;
int lastLine = n / k + (n % k != 0) - 1;

for (int i = 0; i <= lastLine; ++i)
    for (int j = 0; j < k; ++j)
    {
        int ind = getCoords(i, j);
        if (i)
            a[ind] += a[getCoords(i - 1, j)];
        if (j)
            a[ind] += a[getCoords(i, j - 1)];

        if (i && j)
            a[ind] -= a[getCoords(i - 1, j - 1)];
    }

int maxSum = INT_MIN, colGap = k - p;

for (int i = q; getCoords(i, k - 1) < n; ++i)
{
    for (int j = p - 1; j < k; ++j)
    {
        int lineSum = s(i, k - 1) - s(i - q, k - 1),
            colSum = s(lastLine, j) - s(lastLine, j - p),
            intersectSum = s(i, j) - s(i - q, j) + s(i - q, j - p) - s(i,
                j - p);

        if (lineSum + colSum - intersectSum > maxSum)
        {
            maxSum = lineSum + colSum - intersectSum;
        }
    }

    for (int j = colGap; j < k - 1; ++j)
    {
        int hSum = totalSum -
            (s(lastLine, j) - s(lastLine, j - colGap)) +
            (s(i, j) - s(i - q, j) - s(i, j - colGap) + s(i - q, j

```

```

        - colGap));

        if (hSum > maxSum)
        {
            maxSum = hSum;
        }
    }

    fout << maxSum << '\n';
    return 0;
}

```

Observații importante

- Funcția $s(i, j)$ este esențială pentru tratarea corectă a cazurilor de graniță
- Tabla prefix se construiește în ordine crescătoare pentru a asigura disponibilitatea valorilor necesare
- Variabila $\text{colGap} = k - p$ optimizează calculele pentru anumite configurații
- Rezultatul final se scrie în fișierul de ieșire ca un singur număr întreg

Această soluție atinge complexitatea optimă $O(n)$ datorită utilizării ingenioase a sumelor prefix 2D, transformând ceea ce altfel ar fi fost o problemă $O(n^2)$ într-una liniară.